

✓ Skip-gram Word Embedding Training Demo

A self-contained implementation of the Skip-gram model without external libraries.

This notebook demonstrates how word embeddings are learned from context using the Skip-gram architecture. Every mathematical operation is implemented from scratch for educational purposes.

What is Skip-gram?

Skip-gram is a neural network architecture that learns word embeddings by predicting **context words** given a **center word**. For example:

- Input: "cat"
- Output: "the", "sat", "on" (words that appear near "cat")

Through this process, words that appear in similar contexts learn similar vector representations.

✓ 1. Mathematical Helper Functions

First, we implement all the mathematical operations we'll need without using NumPy or any external libraries.

```
import random
import math

def dot_product(vec1, vec2):
    """Compute dot product of two vectors."""
    return sum(v1 * v2 for v1, v2 in zip(vec1, vec2))

def matrix_vector_multiply(matrix, vector):
    """Multiply a matrix by a vector: result[i] = dot(matrix[i], vector)."""
    return [dot_product(row, vector) for row in matrix]

def softmax(logits):
    """
    Compute softmax probabilities from logits.
    softmax(x_i) = exp(x_i) / sum(exp(x_j))
    """
    # Subtract max for numerical stability
    max_logit = max(logits)
    exp_values = [math.exp(x - max_logit) for x in logits]
    sum_exp = sum(exp_values)
    return [exp_val / sum_exp for exp_val in exp_values]

def cross_entropy_loss(probabilities, target_index):
    """
    Compute cross-entropy loss: -log(p[target_index])
    """
    # Clip probability to avoid log(0)
    prob = max(probabilities[target_index], 1e-10)
    return -math.log(prob)

def vector_subtract(vec1, vec2):
    """Element-wise subtraction: vec1 - vec2."""
    return [v1 - v2 for v1, v2 in zip(vec1, vec2)]

def scalar_multiply(scalar, vector):
    """Multiply a vector by a scalar."""
    return [scalar * v for v in vector]

def outer_product(vec1, vec2):
    """
    Compute outer product of two vectors.
    Result is a matrix where result[i][j] = vec1[i] * vec2[j]
    """
```

```

        return [[v1 * v2 for v2 in vec2] for v1 in vec1]

def initialize_random_matrix(rows, cols, scale=0.1):
    """Initialize a matrix with random values."""
    return [[random.uniform(-scale, scale) for _ in range(cols)]
            for _ in range(rows)]

def cosine_similarity(vec1, vec2):
    """Compute cosine similarity between two vectors."""
    dot = dot_product(vec1, vec2)
    mag1 = math.sqrt(sum(v * v for v in vec1))
    mag2 = math.sqrt(sum(v * v for v in vec2))
    if mag1 == 0 or mag2 == 0:
        return 0.0
    return dot / (mag1 * mag2)

print("✓ Helper functions loaded successfully!")

```

✓ Helper functions loaded successfully!

2. Define Corpus and Build Vocabulary

We'll use a small corpus of sentences about cats, dogs, and animals.

```

# Small corpus of text data
corpus = [
    "the cat sat on the mat",
    "the dog sat on the log",
    "cats and dogs are animals",
    "the quick brown fox jumps"
]

# Tokenize: split into words and flatten
words = []
for sentence in corpus:
    words.extend(sentence.lower().split())

print("=" * 70)
print("CORPUS DATA")
print("=" * 70)
print(f"Sentences: {len(corpus)}")
print(f"Total words: {len(words)}")
print(f"Words: {words}")

```

```

=====
CORPUS DATA
=====
Sentences: 4
Total words: 22
Words: ['the', 'cat', 'sat', 'on', 'the', 'mat', 'the', 'dog', 'sat', 'on', 'the', 'log', 'cats', 'and', 'dogs',

```

```

# Build vocabulary: unique words with indices
vocab = sorted(set(words))
word_to_idx = {word: idx for idx, word in enumerate(vocab)}
idx_to_word = {idx: word for word, idx in word_to_idx.items()}
vocab_size = len(vocab)

print("\n" + "=" * 70)
print("VOCABULARY")
print("=" * 70)
print(f"Vocabulary size: {vocab_size}")
print(f"Words: {vocab}")
print(f"\nWord to Index mapping (sample):")
for word in list(word_to_idx.keys())[:5]:
    print(f"  '{word}' -> {word_to_idx[word]}")

```

```

=====
VOCABULARY
=====
Vocabulary size: 16
Words: ['and', 'animals', 'are', 'brown', 'cat', 'cats', 'dog', 'dogs', 'fox', 'jumps', 'log', 'mat', 'on', 'quic

```

Word to Index mapping (sample):

```
'and' -> 0
'animals' -> 1
'are' -> 2
'brown' -> 3
'cat' -> 4
```

3. Generate Training Pairs

For Skip-gram, we create pairs of (**center_word**, **context_word**) where context words are within a window around the center word.

```
def generate_training_pairs(words, word_to_idx, window_size=2):
    """
    Generate (center_word, context_word) pairs for Skip-gram training.
    For each center word, we predict surrounding words within the window.
    """
    pairs = []
    for i, center_word in enumerate(words):
        center_idx = word_to_idx[center_word]

        # Define context window boundaries
        start = max(0, i - window_size)
        end = min(len(words), i + window_size + 1)

        # Extract context words (excluding center word itself)
        for j in range(start, end):
            if j != i:
                context_word = words[j]
                context_idx = word_to_idx[context_word]
                pairs.append((center_idx, context_idx))

    return pairs

window_size = 2
training_pairs = generate_training_pairs(words, word_to_idx, window_size)

print("=" * 70)
print(f"TRAINING PAIRS (window_size={window_size})")
print("=" * 70)
print(f"Total pairs: {len(training_pairs)}")
print("\nSample pairs (center -> context):")
for i in range(min(15, len(training_pairs))):
    center_idx, context_idx = training_pairs[i]
    print(f" {idx_to_word[center_idx]:10s} -> {idx_to_word[context_idx]}")

=====
TRAINING PAIRS (window_size=2)
=====
Total pairs: 82

Sample pairs (center -> context):
the      -> cat
the      -> sat
cat       -> the
cat       -> sat
cat       -> on
sat       -> the
sat       -> cat
sat       -> on
sat       -> the
on        -> cat
on        -> sat
on        -> the
on        -> mat
the       -> sat
the       -> on
```

4. Initialize Weight Matrices

We need two weight matrices:

- **W_{in}**: Input embeddings (vocab_size × embedding_dim)
- **W_{out}**: Output embeddings (vocab_size × embedding_dim)

These are initialized randomly and will be updated during training.

```
embedding_dim = 5 # Dimensionality of word embeddings

# W_in: Input embeddings (vocab_size x embedding_dim)
# Each row is the embedding vector for a word
W_in = initialize_random_matrix(vocab_size, embedding_dim, scale=0.1)

# W_out: Output embeddings (vocab_size x embedding_dim)
# Each row represents the output embedding for a word
W_out = initialize_random_matrix(vocab_size, embedding_dim, scale=0.1)

print("=" * 70)
print("WEIGHT MATRICES INITIALIZED")
print("=" * 70)
print(f"W_in shape: {vocab_size} x {embedding_dim} (input embeddings)")
print(f"W_out shape: {vocab_size} x {embedding_dim} (output embeddings)")
print(f"\nSample initial embedding for 'the' (index {word_to_idx['the']}):")
print(f"  {[f'{x:.4f}' for x in W_in[word_to_idx['the']]])}")

# Save initial embeddings for comparison
initial_W_in = [row[:] for row in W_in] # Deep copy

print("\n✓ Initial embeddings saved for comparison")

=====
WEIGHT MATRICES INITIALIZED
=====
W_in shape: 16 x 5 (input embeddings)
W_out shape: 16 x 5 (output embeddings)

Sample initial embedding for 'the' (index 15):
  ['0.0531', '-0.0442', '0.0569', '-0.0108', '0.0629']

✓ Initial embeddings saved for comparison
```

5. Training Loop

Now we train the Skip-gram model! For each (center, context) pair:

1. **Forward Pass:** Get center word embedding, compute probabilities for all context words
2. **Loss Calculation:** Cross-entropy between predicted and actual context word
3. **Backward Pass:** Compute gradients and update embeddings

Watch the loss decrease over epochs!

```
print("=" * 70)
print("TRAINING")
print("=" * 70)

learning_rate = 0.1
num_epochs = 5

for epoch in range(num_epochs):
    total_loss = 0.0

    # Shuffle training pairs for stochastic gradient descent
    random.shuffle(training_pairs)

    for center_idx, context_idx in training_pairs:
        # -----
        # FORWARD PASS
        # -----
        # Get input embedding for center word
        h = W_in[center_idx] # Hidden layer (embedding vector)

        # Compute output scores (logits) for all words
        logits = matrix_vector_multiply(W_out, h)

        # Apply softmax to get probability distribution
        probabilities = softmax(logits)

        # Compute loss
        loss = cross_entropy_loss(probabilities, context_idx)
```

```

total_loss += loss

# -----
# BACKWARD PASS
# -----
# Gradient of loss w.r.t. output logits
# For cross-entropy + softmax: grad = probabilities - one_hot_target
grad_logits = probabilities[:]
grad_logits[context_idx] -= 1.0

# Gradient w.r.t. W_out (vocab_size x embedding_dim)
# For each word i: grad_W_out[i] = grad_logits[i] * h
for i in range(vocab_size):
    for j in range(embedding_dim):
        W_out[i][j] -= learning_rate * grad_logits[i] * h[j]

# Gradient w.r.t. hidden layer h
# grad_h = W_out^T @ grad_logits
grad_h = [sum(W_out[i][j] * grad_logits[i]
               for i in range(vocab_size))
            for j in range(embedding_dim)]

# Update W_in (only for the center word)
for i in range(embedding_dim):
    W_in[center_idx][i] -= learning_rate * grad_h[i]

avg_loss = total_loss / len(training_pairs)
print(f"Epoch {epoch + 1}/{num_epochs} - Average Loss: {avg_loss:.4f}")

print("=" * 70)
print("\n✓ Training complete!")

```

```

=====
TRAINING
=====
Epoch 1/5 - Average Loss: 2.7728
Epoch 2/5 - Average Loss: 2.7694
Epoch 3/5 - Average Loss: 2.7628
Epoch 4/5 - Average Loss: 2.7457
Epoch 5/5 - Average Loss: 2.7007
=====
✓ Training complete!

```

6. Display Results: Before vs After

Let's compare the embeddings before and after training to see how they've changed.

```

print("=" * 70)
print("EMBEDDING VECTORS (Before and After Training)")
print("=" * 70)

# Show embeddings for a few sample words
sample_words = ["the", "cat", "dog", "sat", "animals"]
sample_words = [w for w in sample_words if w in word_to_idx]

for word in sample_words:
    idx = word_to_idx[word]

    print(f"\nWord: '{word}'")
    print("  Initial embedding:", [f"{x:7.4f}" for x in initial_W_in[idx]])
    print("  Trained embedding:", [f"{x:7.4f}" for x in W_in[idx]])

    # Calculate change magnitude
    changes = [abs(W_in[idx][i] - initial_W_in[idx][i])
               for i in range(embedding_dim)]
    avg_change = sum(changes) / embedding_dim
    print(f"  Average change: {avg_change:.4f}")

```

```

=====
EMBEDDING VECTORS (Before and After Training)
=====

Word: 'the'
  Initial embedding: [' 0.0531', '-0.0442', ' 0.0569', '-0.0108', ' 0.0629']
  Trained embedding: [' 0.4318', '-0.6350', '-0.0358', ' 0.1717', '-0.1072']

```

```

Average change:      0.2830

Word: 'cat'
Initial embedding: [' 0.0462', '-0.0053', '-0.0106', '-0.0349', ' 0.0395']
Trained embedding: [' 0.1385', '-0.2751', ' 0.0760', ' 0.0733', ' 0.0346']
Average change:      0.1124

Word: 'dog'
Initial embedding: ['-0.0409', '-0.0127', '-0.0704', '-0.0584', ' 0.0976']
Trained embedding: [' 0.0585', '-0.2936', '-0.0560', ' 0.0857', ' 0.0730']
Average change:      0.1127

Word: 'sat'
Initial embedding: ['-0.0692', '-0.0292', ' 0.0805', '-0.0363', ' 0.0360']
Trained embedding: ['-0.2324', '-0.6088', ' 0.4528', ' 0.1739', ' 0.3186']
Average change:      0.3216

Word: 'animals'
Initial embedding: [' 0.0470', ' 0.0307', '-0.0943', ' 0.0580', ' 0.0987']
Trained embedding: [' 0.0732', ' 0.0660', '-0.1101', ' 0.2124', ' 0.2302']
Average change:      0.0726

```

7. Word Similarities

Let's check if words appearing in similar contexts have learned similar embeddings using cosine similarity.

```

print("=" * 70)
print("WORD SIMILARITIES (Based on Learned Embeddings)")
print("=" * 70)

# Calculate similarities between some word pairs
word_pairs = [("cat", "dog"), ("the", "on"), ("sat", "jumps"),
              ("cat", "animals"), ("cats", "dogs")]
word_pairs = [(w1, w2) for w1, w2 in word_pairs
              if w1 in word_to_idx and w2 in word_to_idx]

for word1, word2 in word_pairs:
    idx1 = word_to_idx[word1]
    idx2 = word_to_idx[word2]

    initial_sim = cosine_similarity(initial_W_in[idx1], initial_W_in[idx2])
    trained_sim = cosine_similarity(W_in[idx1], W_in[idx2])

    print(f"\n'{word1}' <-> '{word2}':")
    print(f"  Initial similarity:  {initial_sim:7.4f}")
    print(f"  Trained similarity:  {trained_sim:7.4f}")
    print(f"  Change:                {trained_sim - initial_sim:+7.4f}")

```

```

=====
WORD SIMILARITIES (Based on Learned Embeddings)
=====

```

```

'cat' <-> 'dog':
  Initial similarity:  0.4822
  Trained similarity:  0.8788
  Change:              +0.3966

'the' <-> 'on':
  Initial similarity: -0.4482
  Trained similarity:  0.7820
  Change:              +1.2302

'sat' <-> 'jumps':
  Initial similarity:  0.9923
  Trained similarity:  0.3400
  Change:              -0.6523

'cat' <-> 'animals':
  Initial similarity:  0.4335
  Trained similarity:  0.0631
  Change:              -0.3704

'cats' <-> 'dogs':
  Initial similarity:  0.2636
  Trained similarity: -0.1288
  Change:              -0.3924

```

Key Takeaways

1. **Embeddings Update:** The embedding vectors changed from their random initialization based on word co-occurrence patterns

2. **Contextual Similarity:** Words appearing in similar contexts (like "cat" and "dog") should develop more similar embeddings
 3. **Loss Reduction:** The decreasing loss over epochs shows the model is learning
 4. **Simplified Demo:** This is an educational implementation. Real Skip-gram models use:
 - Negative sampling (instead of softmax over full vocabulary)
 - Much larger corpora (millions of words)
 - Higher dimensional embeddings (100-300 dimensions)
 - Optimized implementations (C++/CUDA)
-

Experiment Further!

Try modifying:

- `window_size`: Larger windows capture broader context
- `embedding_dim`: Higher dimensions can capture more information
- `learning_rate`: Affects convergence speed
- `num_epochs`: More training can improve quality
- `corpus`: Add more sentences to see better results!

Start coding or [generate](#) with AI.